МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ЛЬВІВСЬКА ПОЛІТЕХНІКА»

Інститут прикладної математики та фундаментальних наук

Кафедра прикладної математики

Звіт про виконання лабораторної роботи № 3
з дисципліни «Автоматизовані системи тестування програмного продукту»

На тему:
«ТЕСТУВАННЯ WEB API З ВИКОРИСТАННЯМ RESTSHARP
(сервіси restful-booker та Open-Meteo)»

Варіант № 4

Виконав:
студент групи ПМ-31
Погорілець Д. В.
Здано: 05.10.2026

Перевірив:
доцент каф. ПМ
Пабирівський В. В.
Підпис: _____

Львів – 2026

Постановка завдання

Завдання складається з двох частин. У першій потрібно перевірити операції типу CRUD сервісу restful-booker за адресою restful-booker.herokuapp.com. У другій потрібно вибрати будь-яку категорію зі списку відкритих API і перевірити сервіс із неї. Обрано категорію Weather і сервіс Open-Meteo.

Обидві частини вимагають описати поведінку системи мовою Gherkin за методологією BDD і зв'язати опис із кодом через інструмент SpecFlow. Запити до сервісів виконує бібліотека RestSharp. Роботу виконано у середовищі MS Visual Studio, тести запускає MSTest.

Вихідний код розміщено у репозиторії GitHub:
https://github.com/dmytropohorol/LPNUTesting/tree/main/3Lab

Короткі теоретичні відомості

Web API це спосіб звернутися до програми через мережу без участі людини. Клієнт надсилає запит HTTP, сервер відповідає кодом стану і тілом, зазвичай у форматі JSON. Ресурс адресується рядком URL, а потрібна дія задається методом запиту.

Чотири основні дії над даними скорочено називають CRUD: створення, читання, зміна та видалення. У HTTP їм відповідають методи POST, GET, PUT або PATCH і DELETE. Метод PUT замінює ресурс цілком, PATCH змінює лише передані поля.

Код стану повідомляє результат. Група 2xx означає успіх, 4xx помилку у запиті клієнта, 5xx збій на боці сервера. Тому перевірка коду відповіді є першою перевіркою будь-якого тесту API.

RestSharp це бібліотека для платформи .NET, яка складає запит, надсилає його і повертає відповідь у зручному вигляді. Вона сама перетворює об'єкт на JSON перед відправленням і розбирає тіло відповіді назад в об'єкт.

Тестування API відрізняється від Web тестування відсутністю браузера. На екрані перевіряти нема чого, тому тест звіряє код відповіді, вміст тіла та зміну стану на сервері. Прогін через це швидший і стійкіший, бо не залежить від верстки сторінки.

Методологія BDD описує поведінку системи звичайною мовою. Опис зберігається у файлі з розширенням feature мовою Gherkin, а SpecFlow зіставляє кожен його рядок з методом у коді. Докладніше підхід розглянуто у попередній роботі.

Хід роботи

Об’єкти тестування

Сервіс restful-booker створений як майданчик для навчання тестуванню API. Він зберігає бронювання готельних номерів і дозволяє створити, прочитати, замінити, частково змінити та видалити запис. Читання відкрите для всіх, запис потребує маркера доступу. Перелік ресурсів наведено на рисунку 1.

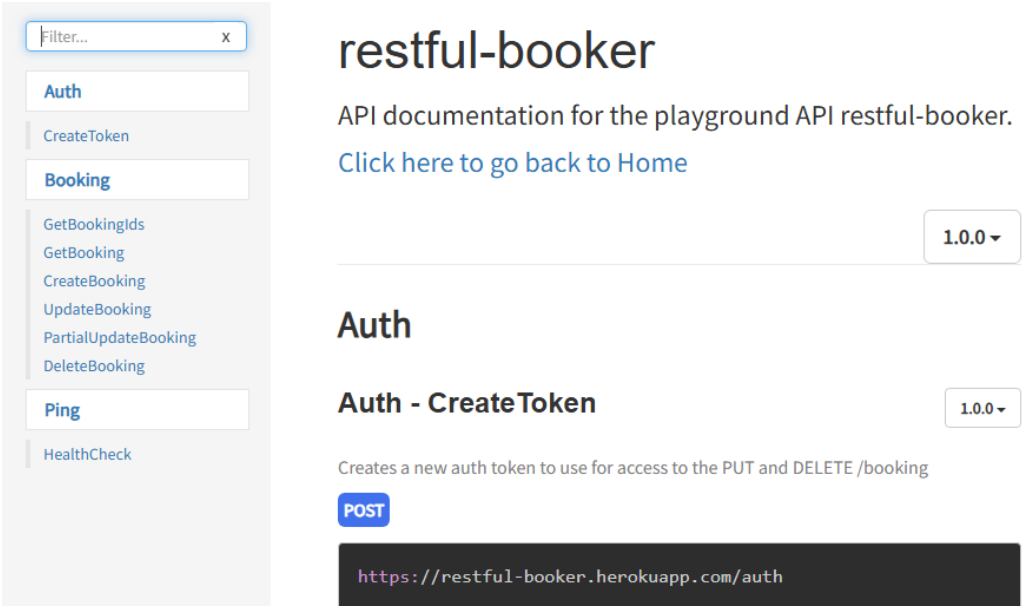


Рисунок 1 – Перелік ресурсів API сервісу restful-booker

Сервіс Open-Meteo віддає прогноз погоди за координатами точки. Авторизації він не потребує, дані лише читаються. Поведінка задається параметрами рядка запиту: координати, набір змінних погоди, одиниці вимірювання, кількість днів та часовий пояс.

Опис поведінки мовою Gherkin

Поведінку описано у трьох файлах. Файл BookingCrud.feature містить операції над бронюванням, BookingAuth.feature перевірку доступу до запису, WeatherForecast.feature прогноз погоди. Основний сценарій першого файла проходить увесь життєвий цикл ресурсу, як показано у лістингу 1.

Лістинг 1 – Опис життєвого циклу бронювання мовою Gherkin

```
Feature: Booking CRUD

Background:
    Given the booking service is available
    And I have an access token

@smoke @crud
Scenario: Booking passes the full life cycle
    When I create a booking
    Then the booking is created and has a number
    When I read the created booking
    Then the booking matches the data I sent
    When I replace the booking with other data
    Then the booking matches the replacement data
    When I delete the booking
    Then the booking is gone
```

Кроки When і Then у сценарії чергуються навмисно: кожна наступна операція має сенс лише після того, як перевірено результат попередньої. Опис не містить ні адрес ресурсів, ні формату тіла запиту, тому його читають без знання програмування.

Клієнти Web API

Звернення до сервісів зібрані у класах BookerClient та WeatherClient зі спільною основою ApiClientBase. Роль цих класів така сама, як у Page Objects під час Web тестування: кроки не знають ні адрес ресурсів, ні формату тіла, тому зміна контракту API зачіпає один файл.

Сервіс restful-booker чекає маркер доступу у заголовку Cookie, а не у звичному для REST заголовку Authorization. Через це вбудований механізм авторизації RestSharp не підходить і заголовок додається вручну.

Налаштування ThrowOnAnyError вимкнено, тому відповідь із кодом поза групою 2xx лишається звичайним результатом. Інакше негативні сценарії не змогли б перевірити ні код, ні тіло відповіді.

Числа у рядку запиту форматуються з явним зазначенням культури. За української культури роздільником дробової частини є кома, і сервіс отримав би широту 49,8397 замість 49.8397.

Реалізація кроків

Кожному рядку сценарію відповідає метод у класах кроків. Стан, який кроки передають один одному, зібрано в об’єкті ApiContext: остання відповідь, маркер доступу та номер створеного бронювання. Статичних полів немає, тому сценарії не залежать від порядку запуску.

Крок перевірки коду відповіді потрібен однаково обом сервісам, тому оголошений один раз у класі CommonSteps. Повторення його у кожному файлі кроків SpecFlow сприйняв би як неоднозначне визначення.

Номер створеного бронювання запам’ятовується у кроці When, бо саме він створює ресурс. Кроки Then лише перевіряють результат і стану сценарію не змінюють.

Запуск тестів

Тести запускаються з Visual Studio через Test Explorer або командою dotnet test. Результат прогону показано у лістингу 2.

Лістинг 2 – Результат прогону тестів Web API

```
Passed CorrectCredentialsReturnAnAccessToken [993 ms]
Passed WrongPasswordDoesNotReturnAToken [650 ms]
Passed UnknownUserDoesNotReturnAToken [650 ms]
Passed WriteOperationWithoutATokenIsForbidden_Replace [1 s]
Passed WriteOperationWithoutATokenIsForbidden_Change [1 s]
Passed WriteOperationWithoutATokenIsForbidden_Delete [1 s]
Passed BookingPassesTheFullLifeCycle [1 s]
Passed CreatedBookingAppearsInTheListOfAllBookings [1 s]
Passed CreatedBookingIsFoundByTheCustomerName [1 s]
Passed PartialUpdateChangesOnlyTheGivenField [1 s]
Passed ReadingABookingThatDoesNotExistReturnsNotFound [775 ms]
Passed CreatingABookingWithoutTheRequiredFieldsIsRejected [897 ms]
Passed WritingToABookingThatDoesNotExistIsRejected_Replace [894 ms]
Passed WritingToABookingThatDoesNotExistIsRejected_Delete [772 ms]
Passed CurrentWeatherIsReturnedForTheGivenCoordinates [209 ms]
Passed DailyForecastReturnsAsManyDaysAsRequested_1 [160 ms]
Passed DailyForecastReturnsAsManyDaysAsRequested_3 [160 ms]
Passed DailyForecastReturnsAsManyDaysAsRequested_16 [161 ms]
Passed ForecastWithTheAutomaticTimeZoneReturnsTheLocalZone [155 ms]
Passed TemperatureIsReturnedInTheRequestedUnit_Celsius [169 ms]
Passed TemperatureIsReturnedInTheRequestedUnit_Fahrenheit [261 ms]
Passed CoordinatesOnTheEdgeOfTheAllowedRangeAreAccepted_Variant0 [167 ms]
Passed CoordinatesOnTheEdgeOfTheAllowedRangeAreAccepted_Variant1 [168 ms]
Passed CoordinatesOnTheEdgeOfTheAllowedRangeAreAccepted_Variant2 [168 ms]
Passed CoordinatesOutsideTheAllowedRangeAreRejected_Variant0 [181 ms]
Passed CoordinatesOutsideTheAllowedRangeAreRejected_Variant1 [162 ms]
Passed CoordinatesOutsideTheAllowedRangeAreRejected_Variant2 [168 ms]
Passed RequestWithoutTheLongitudeIsRejected [162 ms]
Passed ForecastLongerThanTheAllowedRangeIsRejected [163 ms]

Test Run Successful.
Total tests: 29
Passed: 29
Total time: 19.5088 Seconds
```

Набір сценаріїв

Набір складається з 29 тестів: 14 припадає на перше завдання і 15 на друге. Частину з них породжують таблиці Examples у Scenario Outline.

Перше завдання перевіряє життєвий цикл бронювання, появу запису у загальному списку та у вибірці за іменем клієнта і часткову зміну окремого поля. Окремо перевірено відмови: читання відсутнього запису, створення з неповним тілом, запис у відсутній ресурс.

Доступ до запису перевіряють сценарії авторизації. Правильні облікові дані дають маркер на 15 символів, неправильний пароль і невідомий користувач не дають нічого, а заміна, зміна та видалення без маркера не проходять і запису не змінюють.

Друге завдання перевіряє поточну погоду, довжину прогнозу, автоматичний часовий пояс та одиниці температури. Межі допустимих значень перевірено аналізом граничних значень: широта 90 і мінус 90 та довгота 180 приймаються, а 91, мінус 91 і 181 відхиляються з поясненням причини. Так само прогноз на 16 днів приймається, а на 17 відхиляється.

У кожному сценарії з таблицею змінюється рівно один параметр, решта умов стала. Так падіння одразу вказує на причину, а набір не містить повторів тих самих перевірок.

Щоб переконатися, що тести справді перевіряють поведінку, у набір внесено дві навмисні помилки: змінено очікуваний часовий пояс і замінено метод PATCH на PUT. Прогін дав рівно два падіння саме у тих сценаріях, яких ці зміни стосуються. Після повернення вихідного стану всі тести знову пройшли.

Відхилення сервісу від специфікації HTTP

Сервіс restful-booker навмисно містить дефекти, бо створений для навчання. Тестування виявило шість місць, де код відповіді суперечить специфікації.

Перевірка доступності повертає 201 замість 200, хоча нічого не створює. Запит маркера з неправильним паролем повертає 200 і повідомляє про відмову лише полем у тілі, тоді як належав би 401. Створення бронювання з неповним тілом дає 500 замість 400, тобто сервіс подає помилку клієнта як власний збій.

Запис без маркера доступу дає 403 замість 401: код 403 означає, що особа відома, але прав їй бракує, а тут даних для входу немає взагалі. Заміна та видалення відсутнього ресурсу дають 405 замість 404. Видалення наявного бронювання повертає 201, хоча нічого не створює.

Через це негативні сценарії там, де код сервісу суперечить специфікації, перевіряють сам факт відмови, а не конкретне число. Для порівняння Open-Meteo специфікації дотримується: на будь-який некоректний параметр він відповідає кодом 400 і пояснює причину у полі reason.

Висновки

Виконано обидва завдання роботи. Для сервісу restful-booker перевірено операції типу CRUD разом із доступом до запису, для категорії Weather перевірено сервіс Open-Meteo. Набір із 29 тестів проходить повністю за 19,5 секунди.

Запити виконує бібліотека RestSharp. Звернення до сервісів винесені в окремі класи клієнтів, тому кроки тестів не знають ні адрес ресурсів, ні формату тіла, і зміна контракту API зачіпає одне місце.

Поведінку описано мовою Gherkin за методологією BDD, а SpecFlow зв’язує опис із кодом. Опис читається без знання програмування, тому придатний для узгодження очікуваної поведінки із замовником.

Тестування показало, що сервіс restful-booker у шести місцях відповідає кодами, які суперечать специфікації HTTP. Це підтверджує, що перевірка коду відповіді є окремою потрібною перевіркою, а не формальністю.